

Actividad de recuperación día 9



MYSQL CON PHP: BASE DE DATOS, CONSULTAS PREPARADAS Y CREDENCIALES FUERA DEL REPOSITORIO

Fecha	Martes 29 de septiembre de 2026
Aprendiz	Sneider Alvarez Santos
Ficha	3428392
Documento	1016717046
Programa	Desarrollo web — SENA
Proyecto	La Carambola Dorada — sistema de gestión de salones de billar
Entorno	XAMPP — Apache 2.4.58 en el puerto 80, PHP 8.1.25, MariaDB 10.4.32
Publicación	Servidor local XAMPP — http://localhost/plan-mejoramiento-desarrollo-web/

Bitácora del día: qué hice

- 1 Comprobé la versión de PHP con `phpinfo()`: **PHP 8.1.25**, que cumple el mínimo de 8.0 que pide el enunciado.
- 2 `sql/estructura.sql` con las seis tablas, claves foráneas, tipos e índices, y `sql/datos.sql` con **21 productos, 10 clientes y 15 pedidos**.
- 3 `app/config/conexion.php` con elConnection patrón del ejemplo, y las credenciales en un archivo aparte que está en el `.gitignore`, dejando `credenciales.example.php` en su lugar.
- 4 La tabla de productos ahora se llena desde MySQL. `js/datos-prueba.js` desaparece y sus datos se mudaron a `js/ejercicios.js` para que el día 7 no se quedara sin ellos.
- 5 Intenté cinco inyecciones de SQL contra el buscador real y documenté que no ocurre nada: la tabla sigue entera con sus 21 productos.

Estructura de las seis tablas: la quinta entrada es el `proveedor` del formulario del día 2, que también es clave foránea; la sexta es el detalle de cada pedido.

Archivos que creé y archivos que toqué

ARCHIVO	ESTADO	QUÉ HACE
sql/estructura.sql	Nuevo	Crea la base <code>carambola_dorada</code> y las seis tablas con sus claves foráneas e índices.
sql/datos.sql	Nuevo	4 categorías, 4 proveedores, 21 productos, 10 clientes, 15 pedidos y sus 35 líneas de detalle.
app/config/conexion.php	Nuevo	La clase <code>Conexion</code> , con la conexión en una variable estática para abrirla una sola vez.
app/config/credenciales.php	Nuevo, ignorado	El usuario y la clave de mi máquina. En el <code>.gitignore</code> , no se sube.
app/config/credenciales.example.php	Nuevo	La misma forma con valores de mentira. Este sí es el que va al repositorio.
app/modelos/ProductoModelo.php	Nuevo	<code>buscarProductos()</code> y <code>contarProductos()</code> , las dos únicas consultas del proyecto.
productos.php	Nuevo	Reemplaza a <code>productos.html</code> . Pinta las filas con un bucle de PHP y consulta por <code>GET</code> .
phpinfo.php	Nuevo, ignorado	La comprobación del punto 1. También en el <code>.gitignore</code> .
.gitignore	Nuevo	Las credenciales, el <code>phpinfo()</code> y los archivos de respaldo.
app/productos.js	Modificado	Se queda solo con la validación del día 8. La tabla ya no la pinta JavaScript.
js/ejercicios.js	Modificado	Recibió los 16 productos y los 8 pedidos. Los cuatro cálculos no se tocaron.
dashboard.html, componentes.html	Modificado	Solo el enlace del menú: <code>productos.html</code> pasa a ser <code>productos.php</code> .
css/estilos.css	Modificado	<code>.buscador__fila</code> , para el input y el botón Buscar en una sola línea.
js/datos-prueba.js	Borrado	Sus datos pasaron a <code>js/ejercicios.js</code> .
productos.html	Borrado	Lo reemplaza <code>productos.php</code> . No puede seguir siendo <code>.html</code> ; un <code>.html</code> lo entrega Apache sin ejecutar nada.

Los dos archivos borrados quedaron guardados en una carpeta de respaldo fuera del proyecto, por si acaso.

Punto 1. La versión de PHP

XAMPP ya venía instalado en `C:\xampp`, así que no hizo falta instalar nada. La comprobación se hace en `phpinfo.php`, que muestra la versión, el veredicto contra el mínimo de 8.0, la versión del motor de base de datos y si las tres extensiones necesarias están activas.

LO QUE SE COMPRUEBA	RESULTADO
Versión de PHP	<code>8.1.25</code> — cumple el mínimo de 8.0
Apache ejecuta PHP	<code>conf/extra/httpd-xampp.conf</code> línea 20 carga <code>php8apache2_4.dll</code>
Motor de base de datos	<code>MariaDB 10.4.32</code>
Extensiones	<code>pdo_mysql</code> , <code>mysqli</code> y <code>mbstring</code> , las tres activas
Codificación	<code>default_charset=UTF-8</code>

Un aviso que hay que tener presente: el motor de este XAMPP no es MySQL de Oracle sino **MariaDB 10.4.32**, que se presenta como MySQL pero no es. Los dos archivos `.sql` se escribieron sin sintaxis exclusiva de MySQL 8 para que sirvan en los dos: donde hace falta algo versión específica, se usó lo que los dos entienden.

Por qué `phpinfo.php` está en el `.gitignore`: publicar esa página en un servidor real dice la versión exacta de PHP, dónde está instalado y qué extensiones tiene, que es justo el mapa que alguien necesita para buscar un fallo conocido. En clase sirve; en producción no debería existir.

Punto 2. Las seis tablas y los datos

`sql/estructura.sql` crea la base y las seis tablas. Todas son `InnoDB`, porque sin `InnoDB` no hay claves foráneas, que es justo lo que el enunciado pide declarar.

TABLA	COLUMNAS Y TIPOS	CLAVES FORÁNEAS	ÍNDICES
<code>categorias</code>	<code>id INT UNSIGNED</code> , <code>nombre VARCHAR(60)</code> , <code>descripcion</code>	—	<code>uq_categorias_nombre</code>
<code>proveedores</code>	<code>id</code> , <code>nombre VARCHAR(80)</code> , <code>contacto</code> , <code>telefono</code>	—	<code>uq_proveedores_nombre</code>
<code>productos</code>	<code>id</code> , <code>codigo CHAR(7)</code> , <code>nombre VARCHAR(60)</code> , <code>categoria_id</code> , <code>condicion ENUM(...)</code> , <code>existencias SMALLINT UNSIGNED</code> , <code>valor_alquiler DECIMAL(10,2) UNSIGNED</code> , <code>proveedor_id</code> , <code>descripcion</code> , <code>fecha_ingreso DATE</code>	2: a categorías y a proveedores	<code>uq_productos_codigo</code> , <code>idx_productos_nombre</code> y los de las dos foráneas
<code>clientes</code>	<code>id</code> , <code>nombre VARCHAR(80)</code> , <code>documento VARCHAR(20)</code> , <code>telefono</code> , <code>email</code>	—	<code>uq_clientes_documento</code> , <code>idx_clientes_nombre</code>
<code>pedidos</code>	<code>id</code> , <code>cliente_id</code> , <code>fecha DATE</code> , <code>estado ENUM('Pendiente', 'Confirmado', 'Entregado')</code> , <code>total DECIMAL(12,2)</code>	1: a clientes	<code>idx_pedidos_cliente</code> , <code>idx_pedidos_fecha</code> , <code>idx_pedidos_estado</code>
<code>detalle_pedidos</code>	<code>id</code> , <code>pedido_id</code> , <code>producto_id</code> , <code>cantidad SMALLINT UNSIGNED</code> , <code>precio_unitario DECIMAL(10,2)</code>	2: a pedidos y a productos	<code>uq_detalle_pedido_producto</code> , <code>idx_detalle_producto</code>

Cinco claves foráneas en total, comprobadas después de la carga en `information_schema.key_column_usage`. Y se probó que muerden de verdad: intentar insertar un producto con `categoria_id = 999` devuelve el error 1452 y el registro no entra.

Los tipos, y por qué son esos

- `existencias` es `SMALLINT UNSIGNED` con un `CHECK (existencias >= 0)`: no puede haber stock negativo.
- `valor_alquiler` es `DECIMAL(10,2) UNSIGNED` con `CHECK (> 0)`: un alquiler de cero no tiene sentido.
- `codigo` es `CHAR(7)` y único, porque es lo que identifica un producto.
- `condicion` y `estado` son `ENUM`, para que no se pueda guardar un estado inventado.
- El `precio_unitario` se copia en el detalle a propósito: el valor de alquiler de un producto puede cambiar mañana y el pedido de ayer debe seguir cuadrando.

Los datos

El enunciado pide 20 productos, 10 clientes y 15 pedidos. Hay **21, 10 y 15**. Los 21 productos son **los 16 del día 7, los mismos con el mismo código y el mismo valor, más cinco nuevos** (AGT-019, MES-021, REP-020, SOP-022 y TAC-017), para que cambiar el origen de los datos no se note. Se comprobó que los 21 tienen la misma forma y que los 16 del día 7 están todos dentro. Los clientes y los pedidos se apoyan en el nombre de los ocho pedidos del día 7.

Los totales de los pedidos no se escriben a mano: al final de `datos.sql` hay un `UPDATE` que los recalcula como la suma de `cantidad × precio_unitario` de sus líneas de detalle. Los quince cuadran.

Punto 3. La conexión y las credenciales

`app/config/conexion.php` sigue el patrón que da el enunciado: clase final, método estático `obtener()`, y la conexión guardada en `self::$pdo` para que veinte archivos que la pidan abran una sola conexión y no veinte.

A la opción que da el ejemplo le añadí un `catch` que no se traga la excepción: si MySQL está apagado o la clave está mal, la página dice qué pasó en vez de quedar en blanco.

La línea que más importa de todo el archivo es `PDO::ATTR_EMULATE_PREPARES => false`. Con la emulación en `true`, cuando se escribe `$st->bindValue(':limite', $limite, PDO::PARAM_INT)`, PHP le pasa el número ya convertido en texto, MySQL lo envuelve entre comillas y acaba ejecutando `WHERE existencias <= '5'`: el marcador se vuelve **texto** y la comparación de números se resuelve comparando cadenas. Con la emulación apagada el número llega como número. Y como el motor real ya preparó, el punto 5 ni siquiera llega a armarse como SQL.

El archivo de credenciales

ARCHIVO	¿SE SUBE?	POR QUÉ
<code>app/config/credenciales.php</code>	NO	Tiene el usuario y la clave de mi máquina. Está en el <code>.gitignore</code> en la línea <code>app/config/credenciales.php</code> .
<code>app/config/credenciales.example.php</code>	SÍ	La misma forma con valores de mentira. Es lo que sirve de plantilla.
<code>phpinfo.php</code>	NO	También fuera: en un servidor real revela la versión exacta de PHP.

Para trabajar en otra máquina: se copia `credenciales.example.php` como `credenciales.php` y se cambian los cuatro valores. La copia es la que queda fuera del repositorio.

Punto 4. La tabla se llena desde MySQL

`productos.html` se llamó ahora `productos.php`. La extensión no es un detalle: un `.html` lo entrega Apache tal cual, sin ejecutar nada, y entonces no habría manera de preguntarle nada a la base de datos.

Las filas las pinta un bucle de PHP sobre lo que devuelva la consulta, con `htmlspecialchars` en cada celda para que un producto llamado `<script>` no se ejecute. El `data-label` de cada celda, que es lo que convierte la tabla en una lista de tarjetas en el teléfono, lo pone el propio PHP.

El buscador pasó a ser una consulta a MySQL

El buscador del día 8 filtraba en el navegador mientras se escribía. Ahora es un formulario `GET`: el texto viaja en la barra de direcciones y es MySQL el que filtra. Perdió el filtrado en vivo, pero gané algo que para el punto 5 es mejor: el intento de inyección se ve literal en la barra de direcciones, que es la mejor evidencia posible.

El comportamiento del día 8 de que la búsqueda no le importa la tilde **se conservó, pero ahora lo hace la base de datos** y no JavaScript: la columna `nombre` está en `utf8mb4_unicode_ci`, y en esa collation `'extension'` y `'extensión'` son la misma cadena. Se comprobó con las dos formas de escribirlo.

Lo que el día 8 perdió aquí

- **La columna de acciones y la delegación de eventos.** Los botones Editar y Eliminar solo trabajaban sobre el arreglo de JavaScript. Para que funcionaran contra MySQL habría que escribir consultas `UPDATE` y `DELETE`, y el día 9 no las pide. La tabla quedó con seis columnas y de solo lectura.
- **El filtrado mientras se escribe.** Ya está explicado arriba.

Lo que **sí** se conservó del día 8: la validación con el mensaje junto al campo, el menú que responde a la tecla Enter, y los estilos.

El día 7 no se quedó sin datos

`js/ejercicios.js` leía sus productos de `js/datos-prueba.js`. Al borrar ese archivo, sus 16 productos y sus 8 pedidos pasaron al principio de `js/ejercicios.js`, en el mismo formato. Los cuatro cálculos del punto 2 no se tocaron y siguen dando los mismos resultados.

Punto 5. La demostración de la inyección

Se escribieron cinco ataques en el buscador de `productos.php`, contra el buscador real, y se midió lo que pasó: si la página rompía, si devolvía filas, y si la tabla seguía existiendo después.

LO QUE SE ESCRIBIÓ	QUÉ PRETENDÍA HACER	QUÉ PASÓ	RESULTADO
<code>' OR '1'='1</code>	Devolver toda la tabla	0 filas, página normal	No sirvió
<code>TAC-001' OR 1=1 --</code>	Cerrar la comilla y anular la condición	0 filas, página normal	No sirvió
<code>'; DROP TABLE productos; --</code>	Borrar la tabla	0 filas, página normal	No sirvió
<code>'; DELETE FROM productos WHERE 1=1; --</code>	Borrar todas las filas	0 filas, página normal	No sirvió
<code>mesa' UNION SELECT nombre, documento FROM clientes --</code>	Traer los datos de otra tabla	0 filas, página normal	No sirvió

Lo que se ve en pantalla es el aviso de que nada coincide, con el texto del ataque entre comillas, tal como se escribió:

0 de 21 productos coinciden con «'; DROP TABLE productos; --».

La prueba de que el `DROP` no pasó: antes de los cinco ataques la tabla tenía **21 productos**. Después de los cinco, **21 productos**, y la tabla `productos` sigue existiendo en `information_schema.tables`. Si el `DROP` hubiera pasado, el contador habría dado cero.

Por qué no pasó nada

La consulta de `app/modelos/ProductoModelo.php` está escrita así:

```
WHERE p.nombre LIKE :por_nombre
      OR c.nombre LIKE :por_categoria
      OR p.codigo LIKE :por_codigo
```

El texto del usuario va en el **valor** que se enlaza, no dentro de la cadena de SQL. Cuando el motor recibe `'; DROP TABLE productos; --` no ve una sentencia con dos comandos: ve *un solo valor de texto* que empieza con un signo de coma y tiene dos guiones dentro. Para MySQL es un nombre de producto que no existe, y por eso no aparece nada en pantalla.

Los comodines `%` van pegados al valor y no a la consulta, justamente para que nadie pueda escribir sus propios comodines y traerse la tabla entera.

Los criterios de evaluación, punto por punto

CRITERIO	VEREDICTO	CÓMO SE COMPROBÓ
Toda consulta usa <code>prepare()</code> con marcadores	CUMPLE	En el proyecto entero hay 3 llamadas a la base de datos y las tres son <code>prepare()</code> : dos en <code>ProductoModelo.php</code> y una en <code>phpinfo.php</code> . Cero llamadas directas a <code>query()</code> o <code>exec()</code> .
Ninguna consulta con variables concatenadas	CUMPLE	Cero concatenaciones dentro de una cadena SQL. Hay una línea <code>\$patronBusqueda = '%' . \$texto . '%'</code> , y conviene decir qué es: eso arma el valor que después viaja por <code>bindValue</code> . La cadena de SQL de arriba no lo toca. Un buscador bien hecho tiene que poner el comodín ahí, y si lo pusiera en la consulta el usuario podría escribir sus propios comodines.
El archivo de credenciales no está en el repositorio	CUMPLE	<code>app/config/credenciales.php</code> está en la línea 14 del <code>.gitignore</code> , y en su lugar queda <code>credenciales.example.php</code> con valores de mentira.
Las claves foráneas están declaradas	CUMPLE	Cinco declaradas con nombre propio: <code>fk_productos_categoria</code> , <code>fk_productos_proveedor</code> , <code>fk_pedidos_cliente</code> , <code>fk_detalle_pedido</code> y <code>fk_detalle_producto</code> . Confirmadas en <code>information_schema</code> y probadas de verdad: un <code>INSERT</code> con <code>categoria_id = 999</code> es rechazado con el error 1452.
Los scripts SQL están en el repositorio	CUMPLE	<code>sql/estructura.sql</code> y <code>sql/datos.sql</code> . Los dos cargan sin un solo error.

Los problemas que me salieron y cómo los resolví

Un marcador repetido reventó toda la página

La primera versión de `buscarProductos()` repetía el mismo `:texto` tres veces, una por cada `OR`. La página salió con el error `SQLSTATE[HY093]: Invalid parameter number` y la tabla vacía.

La causa es `ATTR_EMULATE_PREPARES => false`: con esa opción MySQL no recibe marcadores con nombre, recibe signos de pregunta, y cada signo se llena con un `bindValue`. Al repetir el nombre, solo se llenaba el primero. Se resolvió con tres marcadores distintos — `:por_nombre`, `:por_categoria` y `:por_codigo` — enlazándoles el mismo valor. El ejemplo del enunciado usa un solo marcador y por eso no le pasa.

Los totales de los pedidos, cálculo dos veces

El `INSERT` de `pedidos` lleva un total, y ese total tendría que ser la suma de `cantidad × precio_unitario` de las líneas de su detalle. Escribes quince números a mano y es fácil equivocarse en uno.

Para no depender de esa cuenta, al final de `datos.sql` hay un `UPDATE` que los **vuelve a calcular** desde el detalle. Se comprobó las dos cosas por separado: cargando el archivo **sin** ese `UPDATE` en una base de prueba, los quince totales escritos cuadran con su detalle (0 descuadres), y cargándolo completo siguen cuadrando. O sea, que hoy el archivo está bien a

mano y lleva la red de seguridad por si alguien cambia una línea de detalle y vuelve a correr el script.

La base tenía 30 pedidos en vez de 15

Este sí lo descubrió al escribir esta bitácora, al contar las filas para escribir la sección de datos, y fue el error más gordo del día. La tabla `pedidos` tenía **30 filas** y `detalle_pedidos` solo 35, todas apuntando a los pedidos 1 a 15. O sea: **15 pedidos de sobra, sin ninguna línea de detalle**.

La causa: el `INSERT` de `pedidos` no lleva el `id`, para que lo ponga MySQL solo. Al cargar `datos.sql` dos veces sin volver a correr antes `estructura.sql`, el primer `INSERT` creó los 15 pedidos y el segundo metió otros 15 con id del 16 al 30. Las otras tablas no se duplicaron porque allí sí hay claves `UNIQUE` que lo impiden.

Se resolvió con el orden correcto, que es el que dice el comentario de `estructura.sql`:

siempre los dos scripts, y primero `estructura.sql`, que empieza con `DROP TABLE IF EXISTS` de las seis tablas. Recargé los dos y quedó en 15 pedidos, 35 líneas, **0 pedidos sin detalle** y 0 totales descuadrados. Además dejé en la bitácora la comprobación que faltaba: no basta con que los totales cuadren, también hay que contar que no haya pedidos huérfanos.

Un fallo de mi propia prueba, dos veces

La prueba del menú con la tecla Enter falló al principio, y el motivo era el mismo de siempre: la estaba corriendo a 1440 px, donde el botón del menú está oculto porque desde 64 rem (1024 px) el menú lateral ya está a la vista (`css/estilos.css:959`, dentro del `@media (min-width: 64rem)` de la línea 942). No hay nada que pulsar, así que el fallo era de la prueba y no del código. Lo segundo fue usar `new KeyboardEvent()`, que llega con `isTrusted = false` y el navegador no lo trata como pulsación real. Se arregló bajando la prueba a 360 px y mandando la tecla con `Input.dispatchEvent`, que es el protocolo del navegador. Añadí una comprobación de que el botón está a la vista, para que un fallo así no pueda volver a pasar en silencio.

Las capturas salieron con 24.459 píxeles de alto

La primera vez, la foto de `phpinfo()` salió con **24.459 píxeles de alto** a 1280 px de ancho, que es la salida entera de `phpinfo()`: más de 1,5 MB de PNG, una imagen que no se puede abrir ni imprimir. La causa es que la toma de pantallas fija la altura de la ventana y luego captura la página completa. Se resolvió recortando cada captura a la caja del elemento que importa, y la del punto 1 quedó en 1005 × 421, que es la ficha con la versión. Ojo: la cifra de 24.459 depende del ancho de la ventana, no es un número fijo.

Un fallo de escritura que dejó código sin revisar

Al escribir los archivos nuevos se me colaron un par de palabras en un idioma que no era el del proyecto. Se buscaron y se quitaron, y se añadió esa búsqueda a la verificación para que no vuelva a pasar. También se quitaron los acentos de los nombres de los productos al escribir el `datos.sql`, lo cual era justo lo contrario de lo que debería: la idea era comprobar que `utf8mb4` aguanta las tildes.

Cómo se verificó

COMPROBACIÓN	CÓMO	RESULTADO
Sintaxis de los 6 archivos PHP	<code>php -l</code> sobre cada uno	Los 6 sin errores
Carga de los dos <code>.sql</code>	<code>mysql -u root < estructura.sql</code> y luego <code>datos.sql</code>	Sin un solo error
Filas por tabla	<code>COUNT(*)</code> en las seis	21 productos, 10 clientes, 15 pedidos, 35 detalles, 4 categorías, 4 proveedores
Claves foráneas	<code>information_schema.key_column_usage</code>	5 declaradas, y el error 1452 al violarlas
Los 15 totales de los pedidos	Comparados contra la suma de su detalle	Los 15 coinciden, y vuelven a cuadrar con el <code>UPDATE</code> final puesto
Pedidos sin líneas de detalle	Los que no aparecen en <code>detalle_pedidos</code>	0 — esta comprobación es la que destapó el error de los 30 pedidos
Los 21 productos en pantalla	Edge real por CDP	21 filas y el rótulo «21 de 21 productos, leídos de MySQL»
El buscador	Seis búsquedas reales	«mesa» 4, «tiza» 2, «TAC-001» 1, «consumible» 6, «extension» 2, «zzz» 0
Las tildes en la base	Buscando con y sin tilde	Las dos formas encuentran «Extensión de taco» y «Extensora rígida»
Los cinco ataques del punto 5	Contra el buscador real	0 filas en los cinco, página normal, y 21 productos antes y después
La validación del día 8	Envío del formulario en blanco	Los cuatro mensajes siguen saliendo junto a su campo
El menú con la tecla Enter	Tecla mandada con <code>Input.dispatchEvent</code>	Abre y cierra, a 360 px
Consola del navegador	Las cuatro páginas, en una sola sesión	Ni un error ni un aviso
Desborde horizontal	320, 360, 768, 1024 y 1440 px en las cuatro páginas	Ninguna desborda
Una sola foto del punto 1	La salida entera de <code>phpinfo()</code>	No se guarda: son 23.896 px de alto y 1,5 MB. Solo la ficha
Sintaxis de los archivos JavaScript	<code>node --check</code>	Los tres sin errores
El HTML de esta bitácora	Validador W3C Nu	0 errores, 0 avisos
El ancho de esta bitácora	360, 768, 1024 y 1440 px	Ninguna tabla se sale del ancho útil
Total	Todo lo anterior, en Edge real	60 comprobaciones, 0 fallos

Evidencia

no - Punto 1 del día 9

8.1.25

VERSIÓN MÍNIMA PEDIDA

8.0.0

RESULTADO

CUMPLE: PHP 8.0 o superior

MOTOR DE BASE DE DATOS

10.4.32-MariaDB

EXTENSIONES NECESARIAS

Las tres activas: pdo_mysql, mysqli y mbstring.

Punto 1. La comprobación de la versión: PHP 8.1.25, que cumple el mínimo de 8.0. Abajo del todo, en la misma página, está la salida completa de `phpinfo()`. El recorte es a propósito: entera son más de 24.000 píxeles de alto y no se puede ni imprimir.

Sneider Alvarez

Buscar en el inventario

Escribe un nombre, una categoría o un código

Buscar

21 de 21 productos, leídos de MySQL

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	VALOR DE ALQUILER
Implementos y consumibles disponibles para alquiler					
AGT-014	Agente de limpieza para mesas	Consumible	Nuevo	12	\$7,500
REP-012	Banda de goma para pocket	Repuesto	Nuevo	9	\$6,000
CON-013	Conector de extensión	Repuesto	En reparación	2	\$9,000
EXT-005	Extensión de taco	Implemento de juego	En reparación	1	\$10,000
SOP-022	Extensora rígida	Implemento de juego	Nuevo	6	\$11,000
GUA-007	Guantes de cuero sintético	Consumible	Semi-nuevo	7	\$22,000
GUA-003	Guantes de fibra	Consumible	Semi-nuevo	6	\$15,000
BOL-004	Juego de bolas de billar	Implemento de juego	Nuevo	3	\$45,000
REP-020	Lodillo de cuero para pocket	Repuesto	Nuevo	10	\$7,000
MES-021	Mesa de billar 7 pies	Mesa de billar	Semi-nuevo	1	\$55,000
MES-008	Mesa de billar 8 pies	Mesa de billar	Nuevo	2	\$60,000
MES-009	Mesa de billar 9 pies	Mesa de billar	Nuevo	1	\$85,000
AGT-019	Paños de microfibra	Consumible	Nuevo	30	\$1,500
PUN-006	Puntería profesional	Implemento de juego	Nuevo	4	\$18,000
REP-011	Repuesto de punta de taco	Repuesto	Nuevo	18	\$3,500
SOP-016	Soporete para taco	Implemento de juego	Nuevo	15	\$5,000
TAC-001	Taco de pool 1,45 m	Implemento de juego	Nuevo	8	\$12,000
TAC-017	Taco de pool 1,55 m	Implemento de juego	Nuevo	4	\$14,000
TIZ-010	Tiza de billar	Consumible	Nuevo	24	\$2,000
TIZ-015	Tiza de Tournament	Consumible	Nuevo	20	\$4,500
TRI-002	Triángulo para bolas	Implemento de juego	Nuevo	5	\$8,000

Cada fila viene de una consulta preparada de `app/modelos/ProductoModelo.php`. El texto que escribas en el buscador viaja como dato, nunca como parte de la consulta.

Punto 4. Las veinte y una filas, pintadas por PHP con un bucle sobre lo que devuelve la consulta preparada. El rótulo de arriba dice de dónde salen.

ario • Sneider Alvarez

Buscar en el inventario

Buscar

2 de 21 productos coinciden con «tiza».

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	VALOR DE ALQUILER
Implementos y consumibles disponibles para alquiler					
TIZ-010	Tiza de billar	Consumible	Nuevo	24	\$2,000
TIZ-015	Tiza de Tournament	Consumible	Nuevo	20	\$4,500

Cada fila viene de una consulta preparada de `app/modelos/ProductoModelo.php`. El texto que escribas en el buscador viaja como dato, nunca como parte de la consulta.

Punto 4. El buscador ya no filtra en el navegador: el texto viaja a MySQL en la barra de direcciones y es la base la que devuelve las dos filas de tiza.

ario • Sneider Alvarez

Buscar en el inventario

Buscar

0 de 21 productos coinciden con «'; DROP TABLE productos; --».

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	VALOR DE ALQUILER
Implementos y consumibles disponibles para alquiler					
Ningún producto coincide con «'; DROP TABLE productos; --».					

Cada fila viene de una consulta preparada de `app/modelos/ProductoModelo.php`. El texto que escribas en el buscador viaja como dato, nunca como parte de la consulta.

Punto 5. El intento de inyección por el buscador real. El texto del ataque se ve entero en el campo y entre comillas en el aviso, porque para MySQL es un nombre de producto que no existe. Cero filas, y la tabla sigue entera.

Bitacora · Sneider Alvarez

Código del producto

Nombre del producto

Escribe el nombre del producto.

Categoría

Seleccione una categoría
 ▼

Selecciona una categoría.

Existencias

Escribe cuántas existencias hay.

Valor de alquiler en pesos

Escribe el valor de alquiler.

Fecha de ingreso al inventario

Proveedor

Descripción

Condición del producto

☒ Nuevo
 ☐ Semi-nuevo
 ☐ En reparación

Registrar producto

Revisa los campos marcados antes de guardar.

La validación del punto 4 del día 8, que sigue viva: los cuatro mensajes junto a su campo.

Cómo se tomaron estas capturas, para que quede claro: se abrió Edge con las medidas reales de cada caso, se mandó la tecla Enter por el protocolo del navegador, y se comprobó el estado de la página en el instante de la captura: las 21 filas, las 2 filtradas, el texto literal del ataque en el campo y el contador de filas en cero. Cada foto está recortada a la caja del elemento que importa. Las capturas son del navegador; no las dibujé yo.

Lo que no hice, y por qué

- **Guardar productos en la base de datos.** El enunciado del día 9 pide leer, no escribir. El botón `Registrar producto` valida los cuatro campos y avisa que el guardado todavía no está conectado. Hacerlo sería un `INSERT` con su formulario de verdad, y no lo piden las líneas de este día.
- **Editar y eliminar.** Consecuencia de lo anterior: la columna de acciones del día 8 se fue y con ella el criterio de delegación de eventos. Recuperarlo pide consultas `UPDATE` y `DELETE`.
- **Las otras cinco tablas en pantalla.** Clientes, pedidos, proveedores y el detalle existen y se pueden ver en `phpMyAdmin`, pero el enunciado solo pide el listado de productos.
- **El filtro en vivo.** Se perdió al pasar la tabla a PHP, y se ganó que el intento de inyección se vea en la barra de direcciones. Era la opción que elegiste cuando te pregunté.

- **Un usuario de MySQL propio para la aplicación.** Se conecta con `root` sin clave, que es el default de XAMPP y sirve para clase. En un servidor de verdad haria falta un usuario con permisos solo sobre esta base.
- **Un repositorio git.** El `.gitignore` está escrito y con la regla correcta, pero el proyecto todavía no es un repositorio. Cuando lo inicialices, la regla empieza a valer sola.

Sneider Alvarez Santos
Aprendiz — Ficha 3428392

Recepción / Instructor

La Carambola Dorada — Actividad de recuperación día 9

Sneider Alvarez Santos · 1016717046 · ficha 3428392